Time Complexity

A practical, accurate guide for DSA learners

Written for students following Striver's A2Z / SDE Sheet (examples in C++, concepts apply to any language)

What you will be able to do after this guide

1. Read any code and work out its time and space complexity step by step.
2. Analyse loops, nested loops, logarithmic loops and recursion (including the Master Theorem).
3. Pick the right algorithm just by looking at the constraints of a problem.
4. Avoid the classic mistakes that even experienced coders make.

Contents

#	Topic
1	What is time complexity?
2	Why it matters: the 10^8 rule and constraints
3	Asymptotic notations: O, Ω, Θ and best/average/worst case
4	Five rules for calculating complexity
5	The common complexity classes
6	Analysing loops (pattern table)
7	Analysing recursion + Master Theorem
8	Space complexity
9	Amortized analysis
10	Cheat sheets: sorting and C++ STL
11	Common mistakes
12	Practice questions with answers
13	A step-by-step method to use in interviews

1. What is time complexity?

Time complexity describes how the running time of an algorithm **grows as the input size grows**. It is *not* the time in seconds. Seconds depend on your laptop, language and compiler. Complexity counts the **number of basic operations** as a function of the input size **n**, and keeps only the part that matters when n becomes large.

> **Key idea**
>
> We care about the **rate of growth**, not the exact count. An algorithm doing 3n + 5 operations and one doing 100n operations are both **O(n)**: if n doubles, the work roughly doubles. But an algorithm doing n^2 operations will quadruple. For large n, that difference decides whether your code passes or gets TLE (Time Limit Exceeded).

Example: counting operations

```
int sum = 0;                      // 1 operation
for (int i = 1; i <= n; i++) {    // runs n times
    sum += i;                     // 1 operation each time
}
cout << sum;                      // 1 operation
// Total ~ n + 2 operations  ->  drop the constant  ->  O(n)
```

What counts as a 'basic operation'?

- Arithmetic (+, −, *, /, %), comparisons, assignments, array access by index, a function call overhead.
- Each of these is treated as **O(1)** (constant time) *as long as the numbers fit in a normal 32/64-bit integer*.
- Be careful: operations like copying a vector, `substr`, or comparing two strings are **not** O(1) — they cost time proportional to their length.

2. Why it matters: the 10^8 rule

In competitive programming and most online judges (LeetCode, Codeforces, GFG), a modern machine does roughly **10^8 (100 million) simple operations per second** in C++ (it is a rule of thumb, not an exact law; heavy operations like divisions, cache misses or STL containers can make it slower). Time limits are usually 1–2 seconds, so your algorithm should do **at most about 10^8 – 10^9 operations**.

Constraint → required complexity

Input size n	Safe complexity	Typical algorithms
n ≤ 10	O(n!) or O(n · n!)	Permutations, brute-force backtracking
n ≤ 20–25	$O(2^n)$ or $O(n \cdot 2^n)$	Subsets, bitmask, recursion with include/exclude
n ≤ 100	$O(n^4)$	Heavy nested loops, small DP
n ≤ 500	$O(n^3)$	Floyd–Warshall, 3-nested loop DP
n ≤ 5,000 – 10,000	$O(n^2)$	Bubble/Selection/Insertion sort, 2D DP, pair checking
n ≤ 10^5 – 10^6	O(n log n) or O(n)	Sorting, binary search, heap, hashing, two pointers, sliding window
n ≤ 10^7	O(n) (tight)	Sieve of Eratosthenes, simple linear pass

Input size n	Safe complexity	Typical algorithms
n up to 10^9 or 10^{18}	O(log n) or O(1) (or O(√n) for 10^{12})	Binary search on answer, fast exponentiation, GCD, math formula

Quick example

n = 10^5 and you write two nested loops: $10^5 \times 10^5 = 10^{10}$ operations ≈ 100 seconds at 10^8/sec. That is a guaranteed TLE. An O(n log n) solution does about $10^5 \times 17 \approx 1.7 \times 10^6$ operations, which is instant. **Always read the constraints first** — they tell you which complexity the problem setter expects.

3. Asymptotic notations

Asymptotic notation describes the behaviour of a function for **very large n**. There are three you must know:

Notation	Name	Meaning	Intuition
O(g(n))	Big-O	f(n) ≤ c · g(n) for large n	**Upper bound** — it will not grow faster than g(n)
Ω(g(n))	Big-Omega	f(n) ≥ c · g(n) for large n	**Lower bound** — it will not grow slower than g(n)
Θ(g(n))	Big-Theta	Both O(g(n)) and Ω(g(n))	**Tight bound** — grows exactly like g(n)

Formal definition of Big-O: f(n) = O(g(n)) if there exist constants c > 0 and n_0 such that f(n) ≤ c · g(n) for all n ≥ n_0. Example: f(n) = $3n^2 + 5n + 7$ is $O(n^2)$ because for n ≥ 1 we have $3n^2 + 5n + 7 \le 15n^2$. It is also (loosely) $O(n^3)$, but we always report the **tightest** bound we can prove.

Best, average and worst case

These describe **which input** you analyse, and they are **separate from** O / Ω / Θ. Each case has its own function, and each can be described with any of the three notations.

Case	Meaning	Linear search for a key in an array of n elements
Best case	The input that makes the algorithm fastest	Key is at index 0 → 1 comparison → O(1)
Average case	Expected cost over all inputs of size n	Key is somewhere in the middle → about n/2 → O(n)
Worst case	The input that makes the algorithm slowest	Key at the end or missing → n comparisons → O(n)

In practice (important for interviews and Striver's sheet)

When someone says "the time complexity is O(n)", they almost always mean the **worst case**, expressed with a tight bound. So in your notes, write the worst case unless the problem says otherwise (e.g. "Quick Sort: O(n log n) average, $O(n^2)$ worst").

Simplification rules for Big-O

- **Drop constants:** O(5n) = O(n), O(n/2) = O(n).
- **Drop lower-order terms:** $O(n^2 + n) = O(n^2)$, O(n + log n) = O(n).
- **Logarithm base does not matter:** $\log_2 n$ and $\log_{10} n$ differ only by a constant factor, so we write O(log n).

4. Five rules for calculating complexity

Rule	What to do	Example
1. Sequential steps ADD	Code blocks one after another: add the costs, keep the biggest	A loop of n followed by a loop of n^2 → $O(n + n^2) = O(n^2)$
2. Nested steps MULTIPLY	Loop inside loop: multiply the iteration counts	Outer n, inner m → $O(n \cdot m)$
3. Different inputs, different variables	If two independent inputs exist, use two letters	Loop over array A (size n) then array B (size m) → $O(n + m)$, **not** $O(n)$
4. Drop constants and small terms	Keep only the dominant term	$3n^2 + 10n + 4 \rightarrow O(n^2)$
5. Function calls cost what the function costs	A call to sort() inside a loop adds $O(n \log n)$ per iteration	for i in n: sort(arr) → $O(n \cdot n \log n)$

Worked example combining the rules

```
void solve(vector<int>& a, vector<int>& b) {   // |a| = n, |b| = m
    sort(a.begin(), a.end());                  // O(n log n)

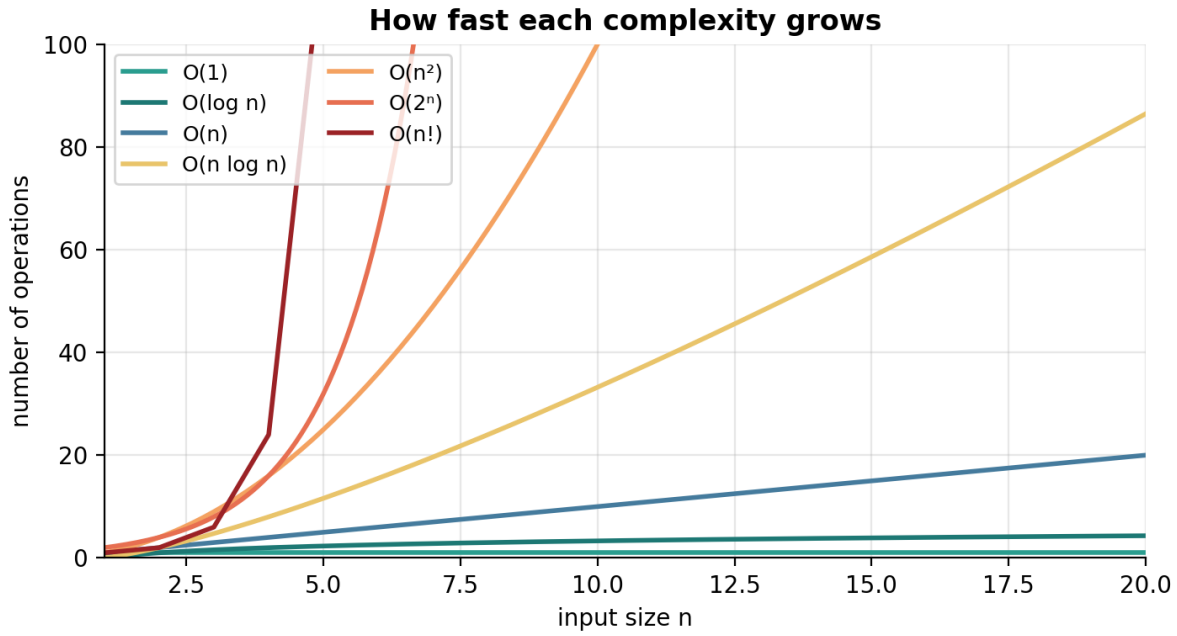
    for (int x : a)                            // O(n)
        cout << x;

    for (int i = 0; i < m; i++)                // outer: m times
        for (int j = 0; j < n; j++)            // inner: n times
            if (a[j] == b[i]) cnt++;           // O(1)  -> O(m * n)
}
// Total = O(n log n) + O(n) + O(m * n)  ->  O(n log n + m * n)
```

5. The common complexity classes

From fastest to slowest. Learn to recognise each one on sight.

Class	Name	Typical example	Feels like
$O(1)$	Constant	Array access a[i], push on stack, hash lookup (average)	Instant, independent of n
$O(\log n)$	Logarithmic	Binary search, Euclid's GCD, fast exponentiation	Input is halved (or divided) every step
$O(\sqrt{n})$	Square root	Checking if n is prime by trying divisors up to √n	Loop runs while $i * i \leq n$
$O(n)$	Linear	Traverse array, find max, Kadane's algorithm, two pointers	One pass over the data
$O(n \log n)$	Linearithmic	Merge sort, heap sort, sort(), n binary searches	Divide in half, do linear work each level
$O(n^2)$	Quadratic	Bubble / selection / insertion sort, all pairs	Nested loop over the same data
$O(n^3)$	Cubic	Naive matrix multiplication, Floyd–Warshall	Three nested loops
$O(2^n)$	Exponential	All subsets, naive recursive Fibonacci	Every element: take it or leave it
$O(n!)$	Factorial	All permutations, brute-force TSP	Arrange n items in every possible order



The chart is capped at 100 operations on the y-axis. Notice how 2^n and n! shoot off the chart almost immediately, while log n barely moves.

Numbers to build intuition (approximate)

Complexity	n = 20	n = 10^5	Time at 10^8 ops/sec for n = 10^5
O(log n)	≈ 4	≈ 17	microseconds
O(n)	20	10^5	1 millisecond
O(n log n)	≈ 86	≈ 1.7 × 10^6	≈ 17 milliseconds
O(n²)	400	10^{10}	≈ 100 seconds → TLE
O(2^n)	≈ 10^6 (1,048,576)	astronomically large	impossible
O(n!)	≈ 2.4 × 10^{18}	astronomically large	impossible

6. Analysing loops

Ask two questions for every loop: **(1) How many times does it run?** and **(2) What does one iteration cost?** Multiply them. Here is a pattern table that covers about 90% of loops you will meet.

Loop pattern	Iterations	Complexity
`for (i = 0; i < n; i++)`	n	O(n)
`for (i = 0; i < n; i += 2) (or i += 5, etc.)`	n / 2	O(n) — constant step is still linear
`for (i = 1; i < n; i *= 2)`	$\log_2 n$	O(log n)
`for (i = n; i >= 1; i /= 2)`	$\log_2 n$	O(log n)
`for (i = 1; i * i <= n; i++)`	√n	O(√n)
`for i in n { for j in n }`	n × n	O(n²)
`for i in n { for (j = 0; j < i; j++) }`	0 + 1 + … + (n−1) = n(n−1)/2	O(n²)

Loop pattern	Iterations	Complexity
`for i in n { for (j = 1; j < n; j *= 2) }`	n × log n	O(n log n)
`for i in n { for j in 100 }`	n × 100	O(n) — inner loop is constant
`for (i = 1; i <= n; i *= 2) { for (j = 1; j <= i; j++) }`	1 + 2 + 4 + … + n ≈ 2n	O(n) — geometric series!
`for (i = 1; i <= n; i++) { for (j = i; j <= n; j += i) }`	n/1 + n/2 + … + n/n = n · H(n)	O(n log n) — harmonic series
`for i in n; then for j in m (separate loops)`	n + m	O(n + m)
`for i in n { for j in m } (nested)`	n × m	O(n · m)

Walk-through 1: dependent inner loop

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < i; j++)
        cout << i << j;
```

When i = 0 the inner loop runs 0 times, when i = 1 it runs 1 time, …, when i = n−1 it runs n−1 times. Total = 0 + 1 + 2 + … + (n−1) = **n(n−1)/2** = (n² − n)/2. Drop the ½ and the lower term → **O(n²)**. This is the pattern behind Selection Sort, Bubble Sort and Insertion Sort.

Walk-through 2: the trick with halving

```
int count = 0;
for (int i = n; i > 0; i /= 2)      // runs log2(n) times
    for (int j = 0; j < i; j++)     // runs i times
        count++;
```

Inner loop runs n, n/2, n/4, … → n(1 + ½ + ¼ + …) < 2n. So the total is **O(n)**, not O(n log n). Many people wrongly multiply “log n × n” here. Always check whether the inner loop's bound *depends on* the outer variable.

Walk-through 3: two pointers / sliding window (amortized loop)

```
int j = 0;
for (int i = 0; i < n; i++) {         // i moves right n times
    while (j < n && cond(i, j))       // j only ever moves right
        j++;
    // ... use the window [i, j)
}
```

This looks like a nested loop, but **j never moves backwards**. Across the entire run, i moves at most n steps and j moves at most n steps, so the total work is n + n = **O(n)**. The same logic applies to the monotonic stack, sliding window maximum and “longest substring without repeating characters”.

Walk-through 4: while loops

```
// Euclid's GCD
int gcd(int a, int b) {
    while (b != 0) { int r = a % b; a = b; b = r; }
    return a;
}
```

Each step replaces (a, b) by (b, a mod b). It can be proven that the value drops by at least half every two steps, so the loop runs **O(log(min(a, b)))** times.

7. Analysing recursion

For recursion, write a **recurrence relation** T(n) = (cost of the calls) + (extra work in one call), then solve it. The easiest way to solve it visually is the **recursion tree**: **Total time = (number of nodes in the tree) × (work done per node)**.

Example A: Factorial (linear recursion)

```
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

T(n) = T(n−1) + O(1) → one call per level, n levels → **Time O(n)**. The call stack holds up to n frames → **Space O(n)**.

Example B: Binary search (recursive)

T(n) = T(n/2) + O(1). The problem halves each time: n → n/2 → n/4 → … → 1, which takes $\log_2 n$ levels → **Time O(log n)**, stack space O(log n).

Example C: Merge sort

```
void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int mid = (l + r) / 2;
    mergeSort(a, l, mid);        // T(n/2)
    mergeSort(a, mid + 1, r);    // T(n/2)
    merge(a, l, mid, r);         // O(n)
}
```

T(n) = 2T(n/2) + O(n). The tree has **log n levels**, and at every level the total merge work is n (e.g. 1 merge of n, or 2 merges of n/2, or 4 merges of n/4 …). So **Time = O(n log n)** in best, average and worst case. Space is O(n) for the temporary array used in merging.

Example D: Naive Fibonacci (the classic exponential)

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

T(n) = T(n−1) + T(n−2) + O(1). Each call makes two calls, so the number of calls roughly doubles per level. A safe upper bound is **$O(2^n)$**; the exact (tight) bound is $\Theta(\varphi^n)$ where $\varphi \approx 1.618$. In interviews, saying $O(2^n)$ is accepted. Space is O(n) — the maximum recursion *depth*, not the number of calls. **With memoization** (DP), every fib(k) is computed only once → **Time O(n), Space O(n)**. This is the foundation of the whole DP section of Striver's course.

Example E: Subsets and permutations (backtracking)

Problem	Recursion shape	Time	Extra space (excluding output)
Print all subsets (pick / not-pick)	2 choices per element, depth n → 2^n leaves, copying a subset costs up to n	$O(n \cdot 2^n)$	O(n) recursion depth
Print all permutations	n! leaves, copying costs n	O(n · n!)	O(n) recursion + O(n) used[] array

Problem	Recursion shape	Time	Extra space (excluding output)
Subset sum with repetition-free pick / not-pick (no printing)	2^n leaves, O(1) work each	$O(2^n)$	O(n)
N-Queens	Place one queen per row; pruned search tree	Roughly O(n!) as an upper bound (pruning makes it much faster in practice)	O(n)

The Master Theorem (shortcut for divide & conquer)

For recurrences of the form $\mathbf{T(n) = a \cdot T(n/b) + \Theta(n^d)}$ where $a \geq 1$, $b > 1$, $d \geq 0$, compare d with $\log_b a$:

Case	Condition	Result	Meaning
1	$d < \log_b a$	$\Theta(n^{\log_b a})$	Leaves dominate
2	$d = \log_b a$	$\Theta(n^d \cdot \log n)$	Every level costs the same
3	$d > \log_b a$	$\Theta(n^d)$	Root dominates

Recurrence	a, b, d	$\log_b a$ vs d	Result	Algorithm
T(n) = T(n/2) + 1	1, 2, 0	0 = 0 (case 2)	O(log n)	Binary search
T(n) = 2T(n/2) + n	2, 2, 1	1 = 1 (case 2)	O(n log n)	Merge sort
T(n) = 2T(n/2) + 1	2, 2, 0	1 > 0 (case 1)	O(n)	Tree traversal / max of array by D&C
T(n) = T(n/2) + n	1, 2, 1	0 < 1 (case 3)	O(n)	Quickselect, average case
T(n) = 4T(n/2) + n	4, 2, 1	2 > 1 (case 1)	$O(n^2)$	—

> **Note**
> The Master Theorem does **not** apply to T(n) = T(n−1) + … (subtract-and-conquer). For those, unroll the recurrence: T(n) = T(n−1) + 1 → O(n); T(n) = T(n−1) + n → n + (n−1) + … + 1 = $O(n^2)$; T(n) = 2T(n−1) + 1 → $O(2^n)$.

8. Space complexity

Space complexity = how much **extra memory** the algorithm needs as n grows. Formally, **Total space = Input space + Auxiliary space**. By convention, most people (and interviewers) mean the **auxiliary space**: the extra variables, arrays, hash maps and recursion stack you create *beyond the input*. If unsure, say which one you are reporting.

Situation	Space
A few integer variables	O(1)
Reversing an array in place (two pointers)	O(1)
Creating a new array / vector / string of size n	O(n)
Hash map / set storing up to n elements	O(n)
2D DP table of size n × m	O(n · m)

Situation	Space
Recursion of depth d	O(d) — the call stack counts as memory!
Merge sort	O(n) (temp array) + O(log n) stack → O(n)
Quick sort	O(log n) expected stack (O(n) in the worst case)
Heap sort / Bubble / Selection / Insertion	O(1)

The recursion stack is the most forgotten part

```
int sum(int n) {          // no arrays, no vectors...
    if (n == 0) return 0;
    return n + sum(n - 1);
}
// Time O(n).  Space O(n) because n function frames sit on the stack.
// The iterative loop version has Space O(1).
```

Space optimization in DP: if dp[i] depends only on dp[i−1] and dp[i−2], keep two variables instead of an array → O(n) space becomes O(1). Striver teaches this as “space optimisation” in every DP problem; now you know the complexity reason behind it.

Time–space trade-off

- Use extra memory to save time: a hash map turns Two Sum from $O(n^2)$ time / O(1) space into O(n) time / O(n) space.
- Use extra time to save memory: recomputing values instead of storing them.

9. Amortized analysis

Sometimes a single operation is occasionally expensive, but the **average cost over a long sequence** is small. That average is the **amortized cost** (it is a guarantee over the sequence, unlike “average case” which depends on random input).

Example — vector push_back (dynamic array). The vector doubles its capacity when full. Most push_backs cost O(1), but when the array is full, all current elements are copied, costing O(n). Over n pushes, the copies cost about $1 + 2 + 4 + \ldots + n \approx 2n$ in total. So n pushes cost O(n) overall, i.e. **amortized O(1) per push_back**, although a single push_back can be O(n).

Other amortized examples: the two-pointer / sliding-window loop from Section 6, the monotonic stack (each element is pushed once and popped at most once), and string append (s += c).

10. Cheat sheets

Sorting algorithms

Algorithm	Best	Average	Worst	Space	Stable?
Bubble sort	O(n)*	O(n^2)	O(n^2)	O(1)	Yes
Selection sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No
Insertion sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick sort	O(n log n)	O(n log n)	O(n^2)	O(log n) stack	No
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No

* Bubble sort is O(n) in the best case only with the “swapped” flag optimisation (stop if a full pass makes no swap). Insertion sort is fast on nearly sorted data. Quick sort’s O(n^2) worst case happens with bad pivots (e.g. already sorted array with the first element as pivot); a random pivot makes it very unlikely.

C++ STL operations

Container / function	Operation	Complexity
vector	a[i] access; push_back / pop_back	O(1); amortized O(1) / O(1)
vector	insert / erase in the middle or at the front	O(n)
vector	find (linear search) / reverse	O(n)
deque	push / pop at both ends	O(1)
list (doubly linked)	insert / erase given an iterator; search	O(1); O(n)
stack / queue	push, pop, top / front	O(1)
set / map (balanced BST)	insert, erase, find, lower_bound	O(log n)
unordered_set / unordered_map (hash)	insert, erase, find	O(1) average, O(n) worst
priority_queue (binary heap)	push, pop; top; build from n elements	O(log n); O(1); O(n)
string	size(); substr(pos, k); s += c; s1 + s2; compare	O(1); O(k); amortized O(1); O(|s1|+|s2|); O(length)
sort()	sort a range of n elements	O(n log n)
binary_search / lower_bound on sorted vector	search	O(log n)
__gcd(a, b)	GCD	O(log(min(a, b)))
next_permutation	one call	O(n)

Other building blocks worth memorising

- Linear search O(n); binary search O(log n) (needs sorted data, or a monotonic yes/no answer space).
- Prime check by trial division: O(√n). Sieve of Eratosthenes: O(n log log n).
- Fast exponentiation (a^b mod m): O(log b).
- Graph traversal BFS / DFS: O(V + E) with an adjacency list.

- Dijkstra with a priority queue: O((V + E) log V).
- Binary tree traversals (inorder / preorder / postorder / level-order): O(n) time; space O(h) where h is the tree height (O(n) worst, O(log n) balanced).
- Trie insert / search of a word of length L: O(L).

11. Common mistakes

1. Counting lines of code instead of growth. A 50-line function can be O(1) and a 3-line function can be O(n²). Always ask: what grows with n?

2. Assuming every nested loop is O(n²). Inner loops bounded by a constant, geometric loops (i *= 2), and two-pointer while-loops can all make nested code O(n) or O(n log n).

3. Forgetting hidden costs. `s.substr()`, `s1 == s2`, copying a vector, passing a vector by value (no &), `v.erase(v.begin())` and `sort()` inside a loop are all non-constant. Passing a vector by value in a recursive function turns O(1) calls into O(n) calls!

4. Using one variable for two inputs. Looping over arrays of sizes n and m is O(n + m) or O(n · m), never just O(n).

5. Ignoring the recursion stack in space complexity. Depth-n recursion uses O(n) space even if you create no arrays.

6. Mixing up "best case" with Ω and "worst case" with O. Cases describe the input; O / Ω / Θ describe the kind of bound. Both can be applied to any case.

7. Treating hash maps as always O(1). It is O(1) on average. A bad hash function or adversarial input can degrade it to O(n) per operation (this is why anti-hash tests exist on Codeforces).

8. Forgetting that numbers are not "size". A loop up to the value N=10^9 is O(N) = 10^9 steps, far too slow. Complexity is measured against the stated variable, so check what n actually is in the constraints.

9. Writing O(n log n) when the log is on the inner structure. Inserting n items into a set costs O(n log n), but n items into an unordered_set is O(n) on average. Match the structure to the claim.

12. Practice questions

Try each one on paper first. Answers with reasoning are given right after.

Q1.

```
for (i = 0; i < n; i++)
    for (j = i; j < n; j++)
        work();
```

Q2.

```
for (i = 1; i <= n; i *= 2)
    for (j = 1; j <= i; j++)
        work();
```

Q3.

```
while (n > 1) n = n / 3;
```

Q4.

```
for (i = 0; i < n; i++)
    for (j = 0; j < 1000; j++)
        work();
```

Q5.

```
void f(int n) { if (n <= 0) return; f(n-1); f(n-1); }
```

Q6.

```
T(n) = 2T(n/2) + 1
```

Q7.

```
for (i = n/2; i <= n; i++)
    for (j = 1; j + n/2 <= n; j++)
        for (k = 1; k <= n; k *= 2)
            work();
```

Q8.

```
Sort n strings, each of length at most m, using sort().
```

Answers

A1. Inner loop runs n, n−1, …, 1 times → n(n+1)/2 → **$O(n^2)$**.

A2. Inner loop runs 1, 2, 4, …, n times. This is a geometric series summing to about 2n → **O(n)** (not n log n).

A3. n is divided by 3 each step → $\log_3 n$ steps → **O(log n)**.

A4. Inner loop is a fixed 1000 iterations → 1000 · n → **O(n)**.

A5. Each call makes two calls with n−1. The number of calls doubles per level: $1 + 2 + 4 + \ldots + 2^n$ → **$O(2^n)$** time, **O(n)** stack space.

A6. Master theorem: a = 2, b = 2, d = 0 → $\log_2 2 = 1 > 0$ → case 1 → **Θ(n)**.

A7. i runs from n/2 to n → n/2 values. j runs 1 to n/2 → n/2 values. k runs log n times. Total = (n/2) · (n/2) · log n → **$O(n^2 \log n)$**.

A8. sort() does O(n log n) comparisons, and each string comparison can cost up to O(m). Total **$O(m \cdot n \log n)$**. (The classic trap: assuming a comparison is O(1).)

Striver-sheet problems: complexity self-check

Problem	Brute force	Optimal
Two Sum	$O(n^2)$ time, O(1) space	Hash map: O(n) time, O(n) space. (Sort + two pointers: O(n log n) time.)
Maximum subarray sum	$O(n^2)$ or $O(n^3)$	Kadane: O(n) time, O(1) space
Merge two sorted arrays	—	Two pointers: O(n + m) time
Search in a sorted array	O(n)	Binary search: O(log n)
Longest consecutive sequence	Sort: O(n log n)	Hash set: O(n) average
Fibonacci / climbing stairs	Recursion: $O(2^n)$	DP: O(n) time; O(1) space with two variables
Subsets / Combination Sum / Permutations	—	Backtracking: $O(n \cdot 2^n)$ / depends on target / $O(n \cdot n!)$

Problem	Brute force	Optimal
Longest substring without repeating characters	$O(n^2)$ or $O(n^3)$	Sliding window: O(n)

13. A step-by-step method to use every time

1. **Name your variables.** Say what n (and m, k …) mean: “n = length of array, m = number of queries”.
2. **Find the dominant work.** Look at loops, recursion and library calls (sort, find, substr, map operations).
3. **Count loops properly.** Straight loop = n; halving or doubling = log n; dependent inner loop = sum the series; two pointers = amortized.
4. **For recursion**, write T(n) and draw the tree: (number of nodes) × (work per node), or use the Master Theorem.
5. **Combine:** add sequential parts, multiply nested parts, then drop constants and lower-order terms.
6. **Do the same for space:** new containers + recursion depth.
7. **Sanity check with constraints:** plug in the maximum n. Is the result below roughly 10^8?
8. **In interviews,** go brute → better → optimal (this is exactly how Striver structures solutions) and state the time and space of each stage out loud.

> **Final tip**
>
> Complexity is a **skill that comes from repetition**. After solving each problem in the sheet, write its time and space complexity in your notes with a one-line reason. After 30–40 problems you will see the patterns instantly. Good luck with your DSA journey!